

C++ Project Documentation

Olli Kankuri 101703439

Bai-Ruffor Bairon 101493891

Krista Voipio 102830783

Nicolas Gavidia-Jokinen 101624039

C++ Project Documentation

Overview	1
FPS Game	2
Known bugs	2
Software structure	2
Class Diagram	2
Instructions for building and using the software	3
Building	3
Using	3
Testing	4
Work log	4
Sprints	4
Division of work	5



Sprint Review

Overview

FPS Game

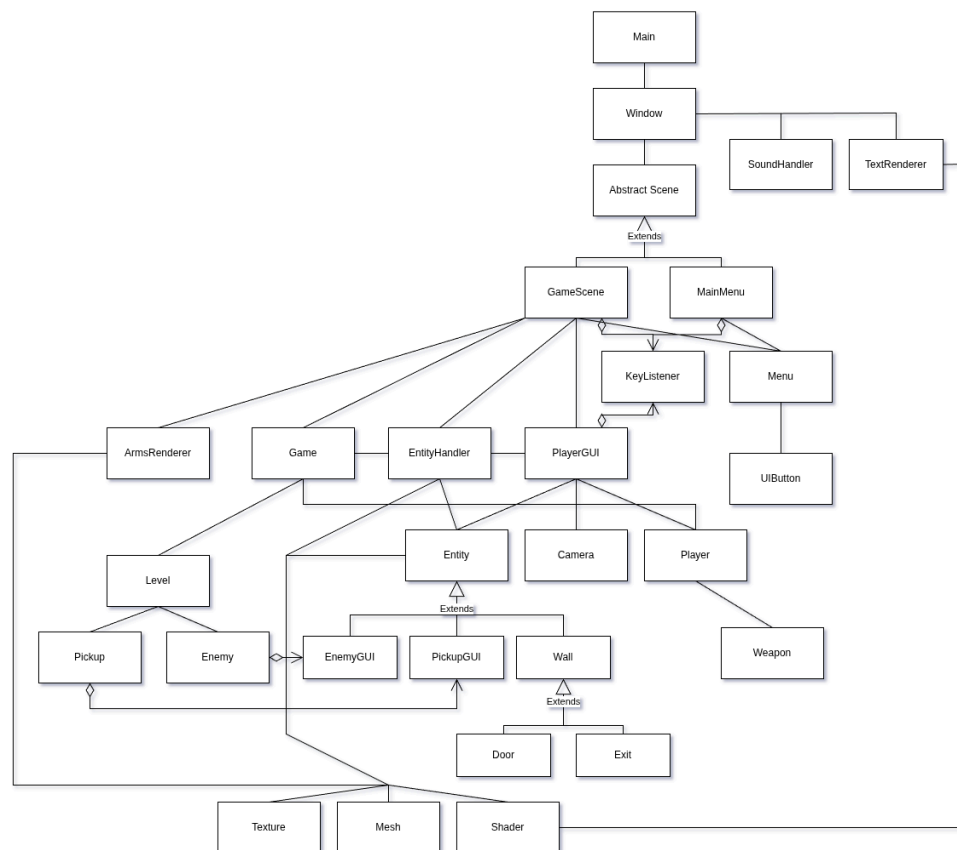
The software is a first-person shooter game, where the player needs to escape Otaniemi. Along the way through Teekkari kylä the player will encounter enemies, who will try to stop the player by dealing damage. The player can hit and shoot the enemies with different weapons, like a fist or cans. The enemies can deal damage to the player until the player dies and has to start over. If the user has saved their progress, they can start from that point if they die or load the game.

Known bugs

- Pausing makes enemies run away from the player
- Enemy sight does not work correctly when looking through another enemy.
- Directional wall squares and doors are not taken into account when pathfinding.
- The reload animation and lockout happens even if the player has 0 stored ammunition or full ammunition.
- Doors don't activate on the first press, but work fine after that.

Software structure

Class Diagram



The connecting lines convey a compositional relationship between classes. A class higher on the graph is composed of connected ones below it. Exceptions were made with the classes where the direction of composition was not obvious through the difference in depth. In these cases, an aggregation arrow was used. Inheritance is shown with an arrow and the “extends” text.

External libraries used are GLFW, GLM, FreeType, STB, OpenGL and SFML. GLFW, GLM, FreeType, and STB are in the /libs directory as git submodules. GLFW is our windowing tool that creates the context in which the game runs. GLM is our maths library from which we use the matrix and vector types. FreeType includes the necessary utilities for text rendering in the game. For image loading, we use STB. OpenGL is the graphics library used to implement the necessary rendering elements. SFML is used for audio.

Instructions for building and using the software

Building

To build the software, the GLFW windowing library submodule needs the following libraries to be downloaded on the computer: wayland, xkb, xll, libxrandr, libxinerama, libxcursor, libxi, and pkg-config.

The project can be built using CMake. Going to the project directory and using “cmake –build build” or “cmake -S . -B build” creates the build directory where the makefile resides. Going into the build directory and running “make” creates the executable. Running the program can be accomplished by inputting “./main” to the terminal in the build directory.

Using

When launched, a starting menu will appear on the screen. The user can choose to play a new game, load a game from an existing game file, or quit. The menu can be navigated with the up and down arrow keys. If the user chooses the “new game” button, the starting level will launch.

In the level there is a player, enemies, pickups and the environment. The user plays as the player and can move around using WASD-keys. The player can shoot enemies with left click if the cursor is pointing at them and they are in range. The user can reload with R, but they cannot move or shoot while they’re doing it. The enemies can be pushed by the player, but they will deal damage if the player is in their range. Pickups can be gathered by walking on them. The user can change levels by opening doors by pressing E. The user cannot go through walls.

The game can be paused by pushing Esc-button, which will present the user a menu. There the game can be continued, saved or loaded and the user can quit and return to the main menu. Saveslots can be changed with the left and right arrow. Choice can be made with Enter.

The game can be won by reaching the end of the final level.

Testing

The software has unit tests for data classes, file reading, and pathfinding. For data classes, the testing was implemented by defining a set of correct outputs for different member functions and comparing them to the actual results from calling them.

We explicitly exclude testing for GUI interaction, user events, and rendering. Instead, the GUI has been tested throughout the implementation of the software. Before committing, each feature would be tested by trying it in action. For example, when creating new interactions between the player and enemies, the one who implemented it would then test it by launching the software and playing the game. If the feature would not work as intended, it would be refined until it works or committed for someone else to fix it.

Work log

At the start of every sprint the team met and planned for the upcoming two weeks. At the start issues would be delegated to those who wanted to do them or knew they could get it done. Along the sprint some issues would not have assignees, so some team members would take them on their own accord. The delegation was also affected by the team consisting of three different operating systems.

Sprints

The project started from meetings, which consisted of brainstorming ideas and planning the upcoming project. During this stage the initial implementation was started.

Sprint 0 consisted of setting the project up for each team member. The libraries were added and the main.cpp file was made for testing compatibility. Classes for upcoming GUI were added. Each member was added to the gitlab of the project and had the project set up on their respective computers.

Sprint 1 had the goal of starting the implementation of GUI and data classes as well as game mechanics. During the sprint data classes for player, weapon, level, game and enemy. Data constant class for player and weapon was added. The basic features for the menu were implemented as well as GUI classes for each data class. In addition game mechanics like collision detection, player moving and picking up pickups were added. The issue board was also set up.

Sprint 2 had the goal of implementing GUI interactions between the user and the software. The HUD and playerGUI class were added for the game to be tested via the player. Steps to later implement working interactions between the player and enemies were made. Tests for data classes were planned but not implemented until sprint 4.

Sprint 3 had the goal of having main interactions and features finalized. The player and enemies can deal damage to each other and both can die. If the player dies the death menu gets displayed. Some audio like background music has been added to the game. Pickups have been implemented. Positions of enemies and pickups can be read from files. The player can switch between levels through intractable doors.

Sprint 4 had the goal of finishing the project documentation, sprites, and testing the software. The code has been documented with doxygen and the sprites have been added. The project documentation was finished. The testing for the software was conducted and bugs have been reported in this documentation. Team members did not keep count of how much time they personally put into the project. Overall the project took an estimated 501 hours.

Division of work

The work was divided along the project like this:

Olli:

- Sprint 0:
 - Planning project structure and features, Adding libraries, camera controls, vertex shader, simple text rendering, plan
- Sprint 1:
 - Window and Scene, scene switching, UIButton, Menu class, GUI constants, KeyListener class, Wall class, EnemyGUI class, PickupGUI class, EntityHandler, Integration of game and menu rendering
- Sprint 2:
 - EntityHandler, Integration of PlayerGUI and player data class, Simple HUD, hit detection, cooldown for shooting, enemies take damage
- Sprint 3:
 - Reloading, stored ammo to weapons, cooldown on attacks, SoundHandler class, fileloading for weapons and pickups, weapon pickups, player dying & death menu, unit tests for player, Enemy and pickup positions load from file, enemies load stats from files
- Sprint 4:
 - Pausing, timed pickups, GUI for timed pickups, downtime for reloading, range to weapons, doxygen for first half of GUI classes, quit button, background color change, loading game menu, winning, Project Documentation (Overview)

Nicolas:

- Sprint 0: Makefile, launch.json & task.json, class for mesh, entity class, component structure, planning project structure and features
- Sprint 1: Collision detection, simple movement, pickup mechanic
- Sprint 2: PlayerGUI class, reading level maps from file
- Sprint 3: Shooting animation
- Sprint 4: Attack animation for enemies, More submodule additions, Class Diagram, Project Documentation (Software structure, Instructions, Testing), Doxygen for some GUI classes

Bai-Ruffor:

- Sprint 0: Planning project structure and features
- Sprint 1: Classes Level, Game, Enemy
- Sprint 2: Enemy movement and pathfinding, player feed and ammo
- Sprint 3: Level switching, interactable doors
- Sprint 4: Savefiles and saveslots, weight to data constants, level and game tests

Krista:

- Sprint 0: Planning project structure and features
- Sprint 1: Added and defined weapon and player class, added data constants
- Sprint 2: Modified Player and Weapon
- Sprint 3: Added enemies attack player, enemies die. Implemented shield, power and speed pickups
- Sprint 4: Pickup sprites, Data Doxygen, Project documentation (Overview, Work log, Testing, Instructions)